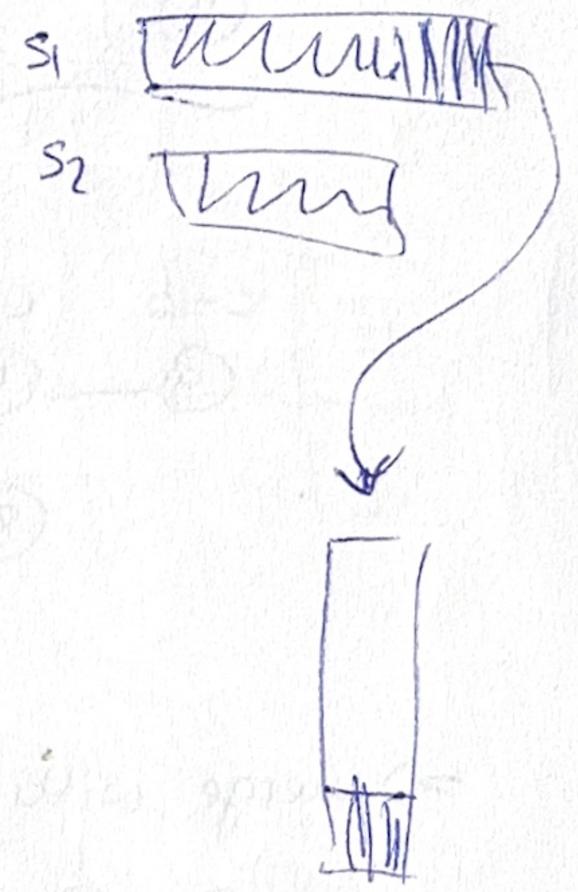
Data Type = domain + interface VS ADT VS DS

DSA - row 5 (2019)

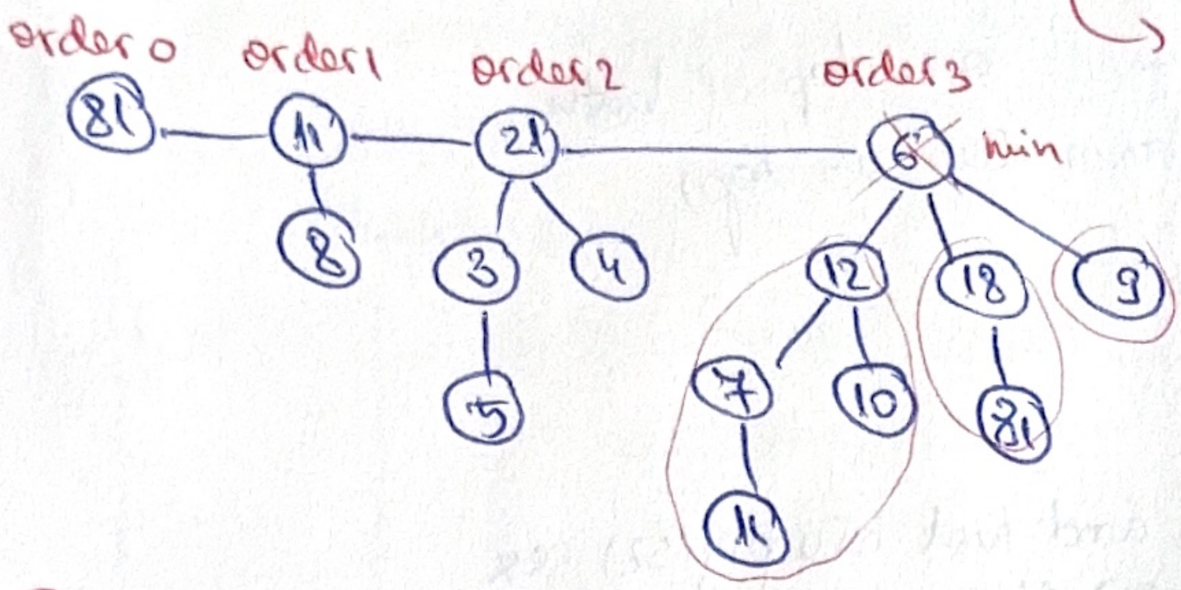
1) → 2 stacks of numbers - min elem. is on top of both
→ create + return a sorted stack (min. elem. on top)

```
function mergeStacks(S1, S2) is
    result ← init()
    aux ← init()
    while not isEmpty(S1) and not isEmpty(S2) ex
        if top(S1) <= top(S2) then
            x ← top(S1)
            pop(S1)
        else
            x ← top(S2)
            pop(S2)
    while not isEmpty(S1) execute
        x ← top(S1)
        pop(S1)
        push(aux, x)
    while not isEmpty(S2) execute
        x ← top(S2)
        pop(S2)
        push(aux, x)
    while not isEmpty(aux) execute
        x ← top(aux)
        pop(aux)
        push(result, x)
    mergeStacks ← result
```



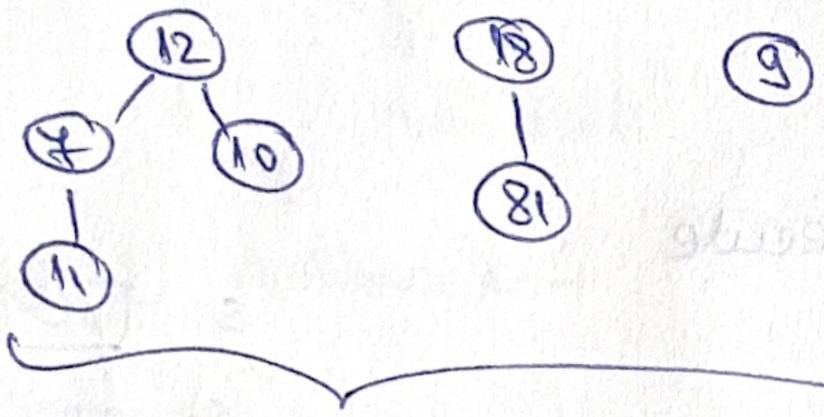
2. a) Priority queue on a binomial heap. = a collection of binomial trees (↑2)
   ↳ merges efficiently
   ↳ order K => 2^K nodes
   ↳ delete root => K bin. trees.
   ↳ 2 trees w the same order can be merged into a binomial tree of order K+1 by setting one to be the LEFT child of the other

pop 1 elem.

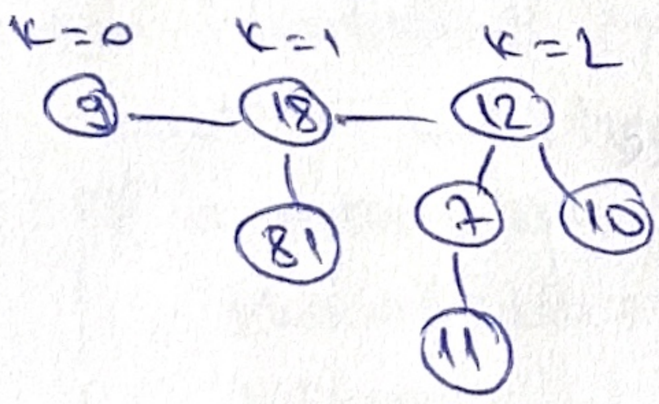


Pop from binomial heap = remove root (min) => seq. of binomial trees
=> transform in bin. heap => merge with the rest of the heap.

=>

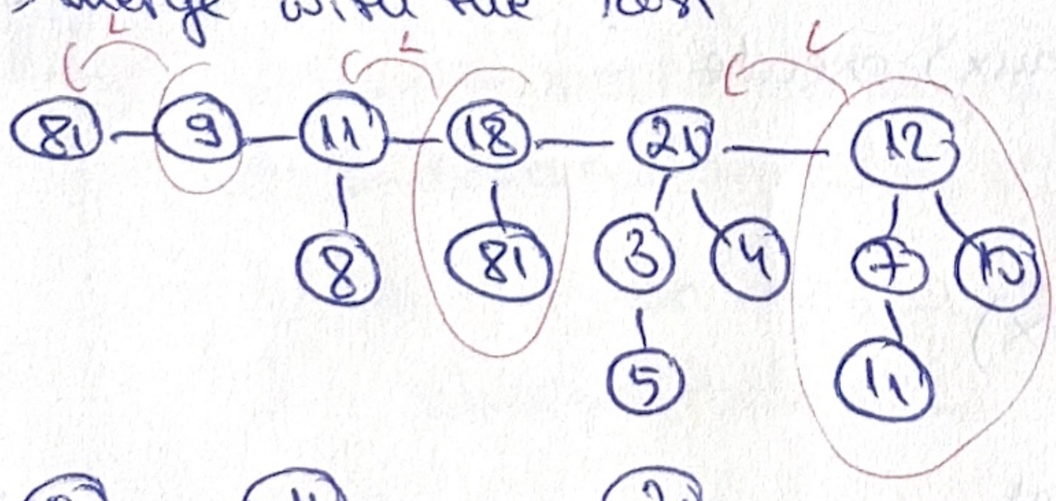


↓ create bin. heap

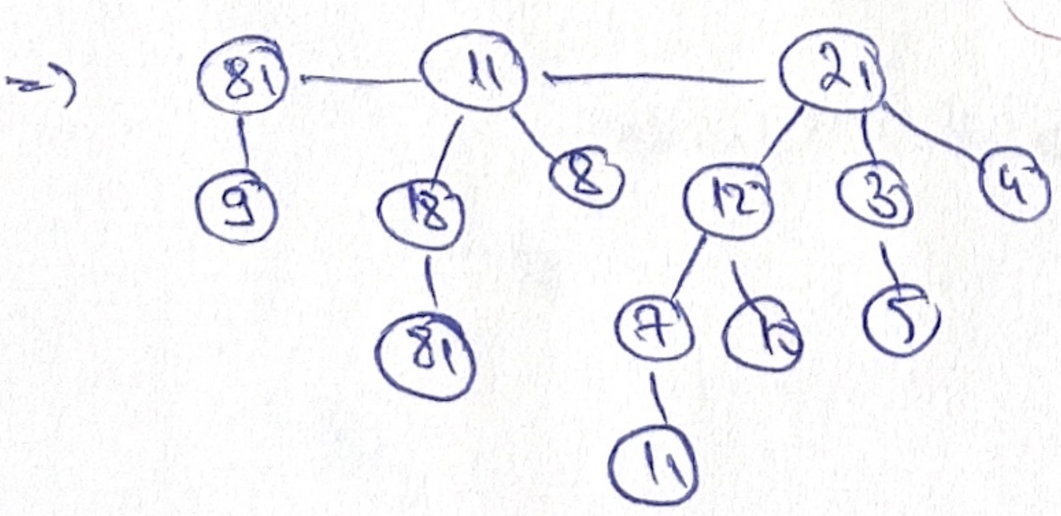


=> merge with the rest

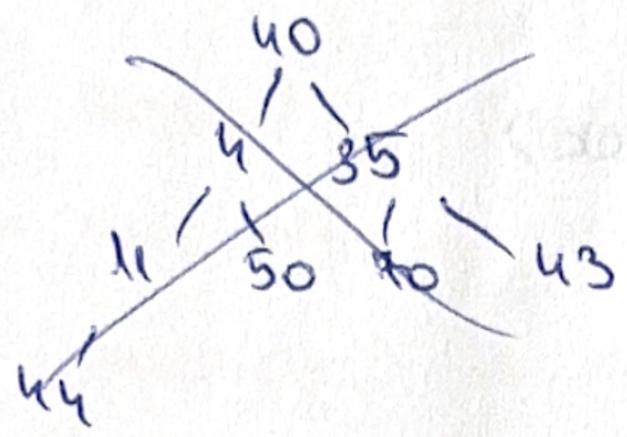
a)



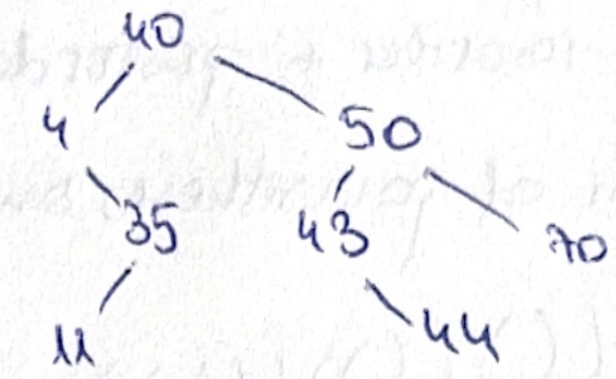
b)



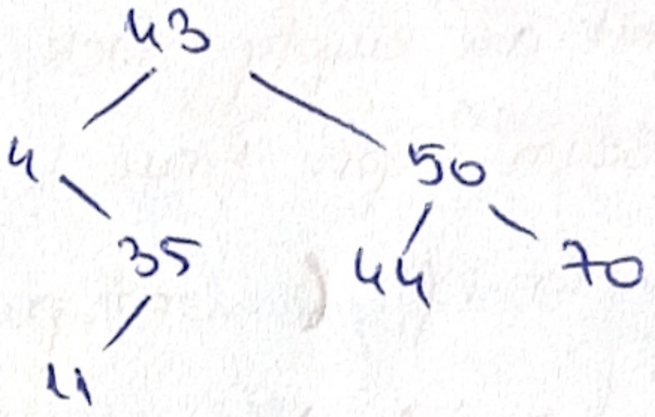
BST
- insert: 40, 4, 35, 11, 50, 70, 43, 44.
- remove 40.



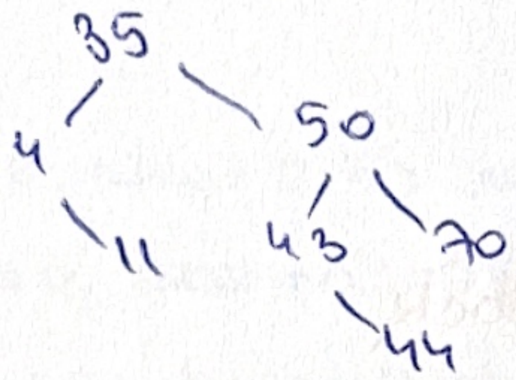
1. replace w predecessor: (35)



2. w successor (43)



2)



c) HT, M=11, collision res. coalesced chaining

insert: 7, 84, 13, 33, 76, 29, 12, 98

remove 7

~~h(7) = 7%11 = 7~~ ✓

~~h(84) = 7~~ ✓

~~h(13) = 2~~ ✓ 2

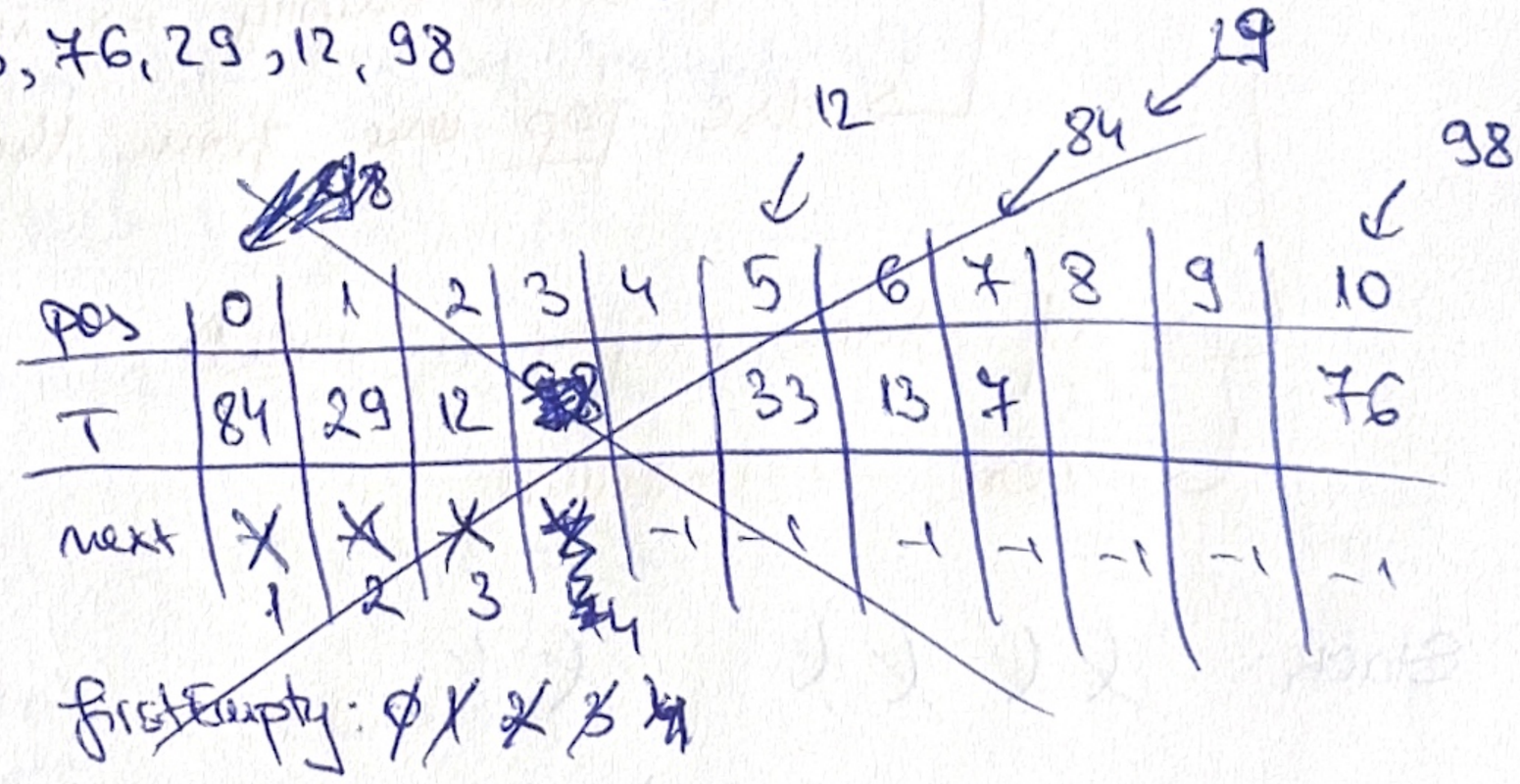
~~h(33) = 0~~ ✓ 0

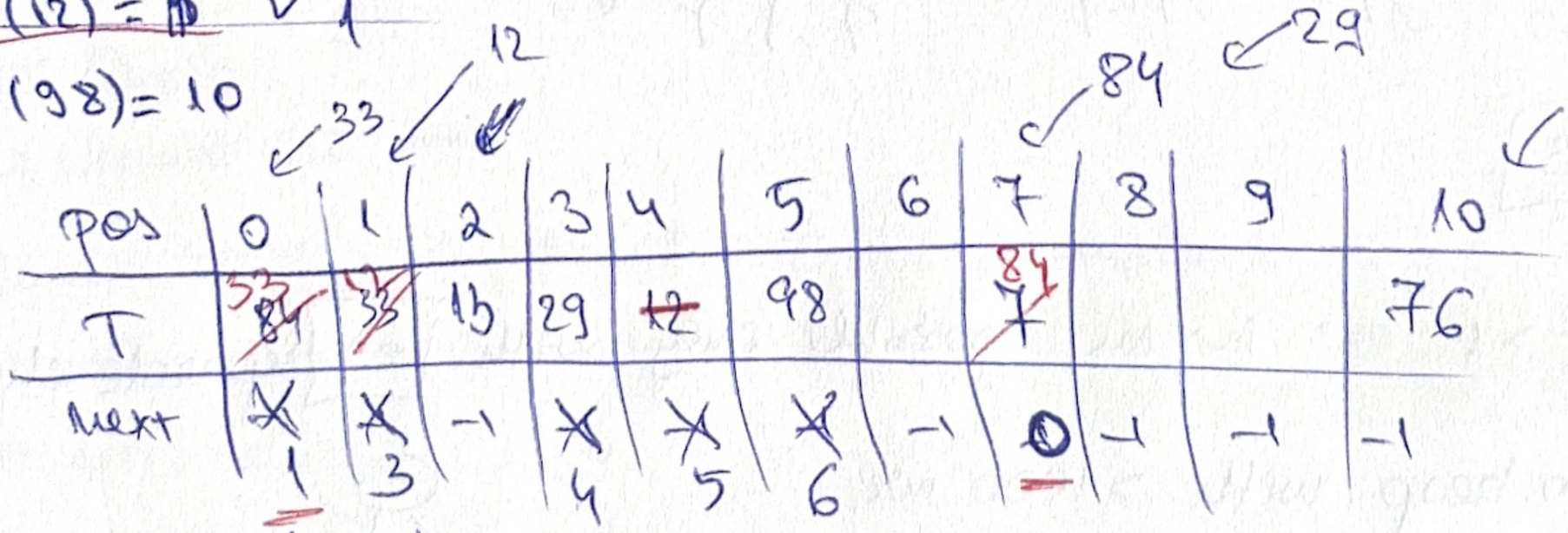
~~h(76) = 10~~ ✓

~~h(29) = 7~~ ✓

~~h(12) = 1~~ ✓ 1

h(98) = 10





Case of removal. ~ If some elem. hashes to the position we are deleting (7), move it there, then continue deletion from the old pos

→ remove 7 from pos 7; 84 hashes to pos 7 => move 84 at pos 7.

→ remove old next(0); 33 hashes to pos 0 => move 33 at pos 0.

→ remove old next(1); 12 hashes to pos 1 => move 12 at pos 1.

old (4); no one hashes there => delete pos 4.

=> final:

P	0	1	2	3	4	5	6	7	8	9	10
T	33	12	13	29	-	98	-	84	-	-	76
N	1	3	-1	4	-1	-1	-1	0	-1	-1	5

1.) From which traversals can we rebuild any BT?

a) preorder + inorder

b) inorder + postorder

2) Max. nr. of parantheses on the stack?

((()(())(()))

Balanced seq. of parantheses algorithm:

1) init. an empty stack

2) scan expr. from L → R

↳ if (=> push onto the stack

↳ if)

↳ if stack empty => invalid seq.

↳ else pop one from the stack → check type of parenthesis

↳ if stack empty => valid

↳ else => invalid.

Stack: ((((((

(((((

((((

(((

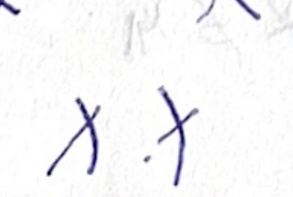
((valid.

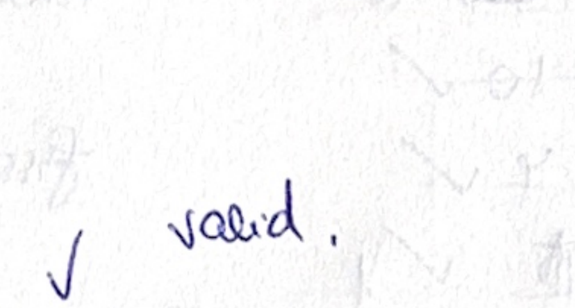
(✓

=> 3

3) $\alpha = \frac{n}{m} > 1 \iff n > m$ possible only with e) separate chaining

4) Bin. min. heap with > 1000 nr.





a. 2nd smallest: level 1 T.

b) 3rd smallest: F (level 2).

c. 4th smallest: level 2. T

? d. 5th smallest: (the pic. is cut)

e. 6th smallest

modify() for Sparse Matrix on BST won't change the str. of the tree?

Sparse matrix: - (line, col, value $\neq 0$)
- sorted by (line, col)
- keep only $\neq 0$ values
- modify() → (4 cases)
  - case I: current = 0 ; new = 0 => do nothing
  - case II: current = 0 ; new $\neq 0$ => insert.
  - case III: current $\neq 0$; new = 0 => remove
  - case IV: current $\neq 0$, new $\neq 0$ => change. (update)

a) current $\neq 0$, new = 0 => remove => changes str.
b) current $\neq 0$, new $\neq 0$ => update => does not change BST str.
c) current = 0, new $\neq 0$ => insert => changes str.
d) current = 0, new = 0 => nothing changes

6) min. nr. of nodes in an AVL tree of height 4?

$$N(h) = 1 + N(h-1) + N(h-2)$$

$$N(4) = 1 + N(3) + N(2) = 1 + 7 + 4 = 12.$$ c) 12

$$N(3) = 1 + N(2) + N(1) = 1 + 4 + 2 = 7$$

$$N(2) = 1 + \underbrace{N(1)}_{2} + \underbrace{N(0)}_{1} = 1 + 2 + 1 = 4$$

4. ADT Queue
↳ circular SLL ~~list~~.

a) Queue repres, implement every op., complexity. THE KEY
→ retain one node only (rear) => front = [q.rear].next
→ $\theta(1)$ compl.

SLCircularList - each node has a successor
- last node = the node whose next is the head.
- retain last node to help insertions.

CSLLNode
info: TElem
next: ↑CSLLNode

Node
info: TElem
next: ↑Node

CSLL
head: ↑CSLLNode

Queue
rear: ↑Node

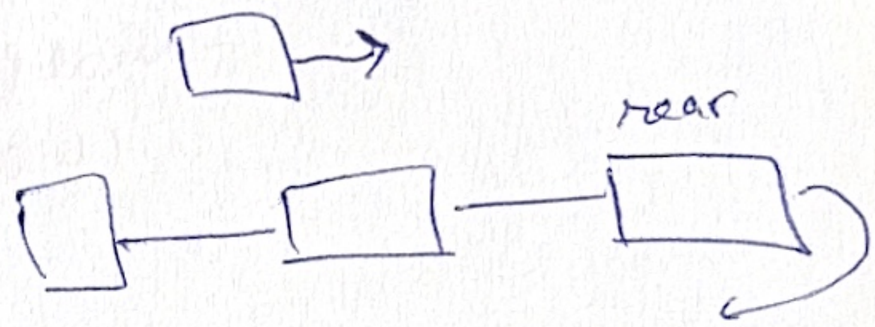
- insertFirst (enqueue)
- deleteLast (dequeue)
- init
- isEmpty
- top (front)

```
subalgorithm init(q) is
    q.rear ← NiL
```

```
function isEmpty(q) is
    isEmpty ← q.rear = NiL
```

→ add to the rear

```
subalgorithm enqueue(q,e) is
    newNode ← allocate
    [newNode].info ← e
    if q.rear = NiL then          // if q is empty
        [newNode].next ← newNode  // circular list => point to itself (the only node in the list)
        q.rear ← newNode          // only node => both front and rear
    else
        [newNode].next ← [q.rear].next   // point to front node (next)
        [q.rear].next ← newNode
        q.rear ← newNode
```



→ remove from front

```
function dequeue(q) is
    if q.rear = Nil then          // q empty
        @throw ....
    front ← [q.rear].next
    dequeue ← [front].info
    if front = q.rear then
        q.rear ← Nil
    else
        [q.rear].next ← [front].next   // skip the front
    deallocate front
```

```
function top(q) is
    if q.rear = Nil then
        @throw ....
    top ← [[q.rear].next].info
```

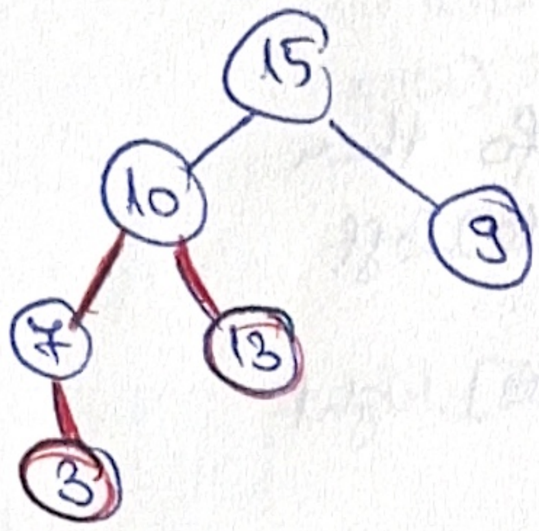
) BST

e_1, e_2 may / may not be in BST.

length of the shortest path between e_1 and e_2.

if either $\nexists e_1$ or $\nexists e_2$ => return -1.

✓ repres. of BST (linked repres, dyn. alloc, without parent / height)
implement op → total complexity.



LCA = 10

distance (10, 3) = 2

distance (10, 13) = 1

=> shortest path = 2 + 1 = 3

ex: 3, 13 => length = 3

least common ancestor (here, 10).

a) **Node**

info : TElem

left : ↑Node

right : ↑Node

BST:

root : ↑Node

b) helper → lowest common ancestor (BST) = the deepest / lowest node that has both c_1 and c_2 as descendants.

```
function LCA(node, c1, c2) is
    while node ≠ NIL ex
        if c1 < [node].info and c2 < [node].info then        // they are both in the left subtree
            node ← [node].left // move down left
        else if c1 > [node].info and c2 > [node].info        // right subtree
            node ← [node].right
        else // we found it (when the paths diverge; one goes left, one goes right!)
            LCA ← node
            return
    LCA ← NIL (one or both ∄)
```

BC: $\Theta(1)$
WC: $\Theta(h)$
} single root-to-leaf path.

helper - distance between two nodes

```
function distance (node, x) is
    d ← 0
    while node ≠ Nil ex
        if x = [node].info then
            distance ← d
            return
        if x < [node).info then
            node ← [node].left
        else
            node ← [node].right
        d ← d+1
    distance ← -1
```

BC: $\Theta(1)$ } single root-to-
WC: $\Theta(h)$ } leaf path

required op:

```
function shortestPath (tree, c1, c2) is
    ancestor ← LCA (tree.root, c1, c2);
    if ancestor = Nil then
        shortestPath ← -1
        return
    d1 ← distance (ancestor, c1)
    d2 ← distance (ancestor, c2)
    if d1 = -1 or d2 = -1 then
        shortestPath ← -1
    else
        shortestPath ← d1 + d2
```

BC: $\Theta(1)$ } h = height of BST
WC: $\Theta(h)$ }

T: $O(h)$

h = height of BST
$h = \log_2 n$ if BST balanced
$h = n$ if BST skewed